

特别地, SQL 是一种高度非过程化的语言, 用户不必请求数据库管理员为其建立特殊的存取路径, 存取路径的选择由关系数据库管理系统的优化机制来完成。例如, 在一个存储有千百万条记录的关系中查找符合条件的某一条记录或某一些记录, 从原理上讲可以有多种查找方法。例如, 可以顺序扫描这个关系, 也可以通过某一种索引来查找。不同的查找路径(也称为存取路径)的效率是不同的, 有的完成某一个查询可能很快, 有的可能极慢。关系数据库管理系统研究和开发了查询优化方法, 系统可以自动选择较优的存取路径, 以提高查询效率。

## 2.3 关系的完整性

关系模型的完整性约束是对关系的某种约束条件。也就是说关系的值随着时间变化时应该满足一些约束条件, 这些约束条件实际上是现实世界的要求。任何关系在任何时刻都要满足这些语义约束。

关系模型中有三类完整性约束: 实体完整性(entity integrity)、参照完整性(referential integrity)和用户定义的完整性(user-defined integrity)。其中实体完整性和参照完整性是关系模型必须满足的完整性约束, 被称作是关系的两个不变性, 应该由关系系统自动支持。用户定义的完整性是应用领域需要遵循的约束条件, 体现了具体领域中的语义约束。

### 2.3.1 实体完整性

关系数据库中每个元组应该是可区分的, 是唯一的。这样的约束条件用实体完整性来保证。

**规则 2.1 实体完整性约束** 若属性(指一个或一组属性) $A$  是基本关系  $R$  的主属性, 则  $A$  不能取空值(null value)。所谓空值就是“不知道”或“不存在”或“无意义”的值。有关空值的处理将在第3章 3.5 节空值的处理中详细讲解。

例如, 学生(学号, 姓名, 性别, 出生日期, 主修专业)关系中“学号”为主码并用下划线标识, 则学号取值唯一, 且不能取空值。

按照实体完整性约束的规定, 如果主码由若干属性组成, 则所有这些属性都不能取空值。例如, 学生选课(学号, 课程号, 成绩, 开课学期, 教学班)关系中, “学号, 课程号”为主码, 则“学号”和“课程号”两个属性都不能取空值。

对于实体完整性约束说明如下:

① 实体完整性约束是针对基本关系而言的。一个基本表通常对应现实世界的一个实体集。例如, “学生”关系对应学生的集合。

② 现实世界中的实体是可区分的, 即它们具有某种唯一性标识。例如, 每个学生都是独立的个体, 是不一样的。

③ 相应地, 关系模型中以主码作为唯一性标识。

④ 主码中的属性不能取空值, 如果取了空值, 就说明存在某个不可标识的实体, 即存在

不可区分的实体,这与②相矛盾。因此这个规则称为实体完整性。

### 2.3.2 参照完整性

现实世界中的实体之间往往存在某种联系,在关系模型中实体及实体间的联系都是用关系来描述的,这样就自然存在着关系与关系间的引用。下面先来看三个示例。

[例 2.1]“学生”实体和“专业”实体可以用下面的关系模式来表示。其中,“专业”实体有 2 个候选码“专业编号”和“专业名”,可以选其中一个为主码,即“专业名”。

学生(学号,姓名,性别,出生日期,主修专业)

专业(专业名,专业编号) /\* 专业名、专业编号都是候选码,取专业名为主码 \*/

这两个关系之间存在着属性的引用,即“学生”关系引用了“专业”关系的主码“专业名”。显然,学生关系中的“主修专业”值必须是确实存在的专业名,即“专业”关系中有该专业的记录。也就是说,“学生”关系中“主修专业”属性的取值需要参照“专业”关系中“专业名”属性的取值。

[例 2.2]“学生”“课程”关系以及两者之间的多对多联系可以用如下三个关系模式表示。

学生(学号,姓名,性别,出生日期,主修专业)

课程(课程号,课程名,学分,先修课)

学生选课(学号,课程号,成绩,开课学期,教学班)

这三个关系模式之间也存在着属性之间的引用,即“学生选课”关系引用了“学生”关系的主码“学号”和“课程”关系的主码“课程号”。同样,“学生选课”关系中的“学号”值必须是确实存在的学生的学号,即“学生”关系中有该学生的记录;“学生选课”关系中的“课程号”值也必须是确实存在的课程的编号,即“课程”关系中有该课程的记录。换句话说,“学生选课”关系中某些属性的取值需要参照其他关系的属性取值。

不仅两个或两个以上的关系间可能存在引用关系,同一关系的内部属性间也可能存在引用关系。

[例 2.3]在课程(课程号,课程名,学分,先修课)关系中,“课程号”属性是主码,“先修课”属性表示选修该门课程之前需要完成先修课程的课程号。它引用了本关系的“课程号”属性,即“课程号”必须是确实存在的课程的编号。

这三个例子说明关系与关系之间、同一关系内部属性之间存在相互引用、相互约束的情况。下面先引入外码的概念,然后给出表达关系之间相互引用约束的参照完整性的定义。

**定义 2.5** 设  $F$  是基本关系  $R$  的一个或一组属性,但不是关系  $R$  的码,  $K_s$  是基本关系  $S$  的主码。如果  $F$  与  $K_s$  相对应,则称  $F$  是  $R$  的外码(foreign key),并称基本关系  $R$  为参照关系(referencing relation),基本关系  $S$  为被参照关系(referenced relation)或目标关系(target relation)。关系  $R$  和  $S$  不一定是不同的关系。如图 2.2 所示。

显然,目标关系  $S$  的主码  $K_s$  和参照关系  $R$  的外码  $F$  必须定义在同一个(或同一组)域上。



图 2.2 参照关系与被参照关系示意图

在例 2.1 中，“学生”关系的“主修专业”属性与“专业”关系的主码“专业名”相对应，因此“主修专业”属性是“学生”关系的外码。这里“专业”关系是被参照关系，“学生”关系为参照关系。如图 2.3(a) 所示。

在例 2.2 中，“学生选课”关系的“学号”属性与“学生”关系的主码“学号”相对应，“学生选课”关系的“课程号”属性与“课程”关系的主码“课程号”相对应，因此“学号”和“课程号”是“学生选课”关系的外码。这里“学生”关系和“课程”关系均为被参照关系，“学生选课”关系为参照关系。如图 2.3(b) 所示。

在例 2.3“课程”关系中，“先修课”属性与本身的主码“课程号”属性相对应，因此“先修课”是外码。这里，“课程”关系既是参照关系也是被参照关系。如图 2.3(c) 所示。



图 2.3 关系的参照图

需要指出的是，外码并不一定要与相应的主码同名，如例 2.1 中“专业”关系的主码为“专业名”，外码为“主修专业”；例 2.3 中“课程”关系的主码为“课程号”，外码为“先修课”。

在实际应用中为了便于识别，当外码与相应的主码属于不同关系时，往往给它们取相同的名字，例如“学生选课”关系中的外码“课程号”与“课程”关系中的主码取相同属性名。

参照完整性约束就是定义外码与主码之间的引用规则。

**规则 2.2 参照完整性约束** 若属性(或属性组) $F$ 是基本关系 $R$ 的外码，它与基本关系 $S$ 的主码 $K_S$ 相对应(基本关系 $R$ 和 $S$ 不一定是不同的关系)，则对于 $R$ 中每个元组在 $F$ 上的值必须：

- ① 或者取值( $F$ 的每个属性值均为空值)。
- ② 或者等于 $S$ 中某个元组的主码值。

例如，对于例 2.1，“学生”关系中每个元组的“主修专业”属性只能取下面两类值：

- ① 空值，表示该学生尚未选择主修专业。
- ② 非空值，这时该值必须是“专业”关系中某个元组的“专业名”值，表示该学生不可能选

一个不存在的专业。即被参照关系“专业”中一定存在一个元组，它的主码值等于该参照关系“学生”中的外码值。

对于例 2.2，按照参照完整性约束，“学号”和“课程号”属性也可以取两类值：空值或目标关系中已经存在的值。但由于这两个属性是“学生选课”关系中的主属性，按照实体完整性约束，它们均不能取空值，所以“学生选课”关系中的“学号”和“课程号”属性实际上只能取相应被参照关系中已经存在的主码值。

在参照完整性约束中， $R$  与  $S$  可以是同一个关系。

例如对于例 2.3，按照参照完整性约束，“先修课”属性值可以取两类值：

- ① 空值，表示该门课程不存在先修课。
- ② 非空值，这时该值必须是本关系中某个元组的课程号。

### 2.3.3 用户定义的完整性

任何关系数据库系统都应该支持实体完整性和参照完整性，这是关系模型所要求的。除此之外，不同的关系数据库根据其应用场景不同，往往还需要一些特殊的约束条件。用户定义的完整性就是针对某一具体关系数据库的约束条件，它反映某一具体应用所涉及的数据必须满足的语义要求。例如，某个属性必须取唯一值，某个非主属性不能取空值等。例如，在例 2.1 的“学生”关系中，若要求学生必须有姓名，则可以定义“姓名”属性不能取空值；“学生选课”关系中“成绩”的取值范围可以定义为 0~100 等。

关系模型应提供定义和检验这类完整性约束的机制，以使用统一的系统的方法处理它们，而不需要由应用程序承担这一功能。

## 2.4 关系代数

关系代数是一种抽象的查询语言，它用对关系的运算来表达查询。

任何一种运算都是将一定的运算符作用于一定的运算对象上，得到预期的运算结果。所以运算对象、运算符、运算结果是运算的三大要素。

关系代数的运算对象是关系，运算结果亦为关系。关系代数用到的运算符包括两类：集合运算符和专门的关系运算符，如表 2.4 所示。

按运算符的不同，关系代数的运算可分为传统的集合运算和专门的关系运算两类。其中，传统的集合运算将关系看成元组的集合，其运算是从关系的“水平”方向，即行的角度来进行；专门的关系运算不仅涉及行，而且涉及列。比较运

表 2.4 关系代数运算符

| 运算符      |           | 含义   |
|----------|-----------|------|
| 集合运算符    | $\cup$    | 并    |
|          | $-$       | 差    |
|          | $\cap$    | 交    |
|          | $\times$  | 笛卡儿积 |
| 专门的关系运算符 | $\sigma$  | 选择   |
|          | $\Pi$     | 投影   |
|          | $\bowtie$ | 连接   |
|          | $\div$    | 除    |